

Application Correction, Refactoring & Test Automation

A comprehensive 5-stage pedagogical learning journey designed to empower Pavel with elite agentic pair programming skills in Google Antigravity (AGY) using the Node Hierarchy Manager.

1. Executive Summary & Pedagogical Blueprint

Welcome to the **Antigravity (AGY) Mastery Journey**. Antigravity is an autonomous **agentic pair-programming platform** equipped with deep codebase cartography, terminal execution, AST-aware diffing, background tasks, and multi-subagent orchestration. This curriculum is specifically tailored for Pavel to master how to leverage AGY for isolating root causes, fixing complex bugs, enforcing test suites, and refactoring full-stack TypeScript applications.

■ Core Competency 1: Root-Cause Debugging Isolate subtle async race conditions, recursive tree cycles, and strict TypeScript/lint violations using AST-aware cartography rather than trial-and-error guesswork.	■ Core Competency 2: Automated Verification Loops Direct AGY to build test harnesses (Vitest, React Testing Library), enforce test-driven fixes (Red-Green-Refactor), and run continuous regression suites.
■ Core Competency 3: Multi-Subagent Orchestration Spawn specialized subagents (Security Auditor, Performance Profiler, Test Writer) to perform parallel codebase reviews and refactor large subsystems safely.	■ Core Competency 4: Guardrails & Quality Gates Configure project-scoped operational rules in AGENTS.md and author domain skills that permanently prevent regressions and enforce standards.

2. Case Study Architecture: Node Hierarchy Manager

All modules in this curriculum utilize the **Node Hierarchy Manager** codebase as an authentic testbed. The application combines deep recursive UI state, Supabase database persistence, and external AI curation workflows:

Subsystem	Key Files / Components	Engineering & Testing Challenges
Tree Engine & State	src/components/NodeTree.tsx src/components/NodeItem.tsx	Recursive tree traversal, infinite cycle prevention, drag-and-drop reordering, Supabase state synchronization.
Modal Workflows & Curation	src/components/CurationModal.tsx src/components/AIQueryRefinementModal.tsx	Multi-step modal state machines, async payload transformations, NotebookLM artifact generation hooks.
Database & Persistence Layer	supabase_migration_*.sql src/services/NotebookLMService.ts	Cascade deletes, row-level security (RLS), stored procedures, SQL triggers, and multi-tenant isolation.
Build & Type Integrity	tsconfig.app.json eslint.config.js	TypeScript strict null-checking, React 19 compiler compatibility, ESLint boundary compliance.

3. The 5-Stage Progressive Learning Curriculum

The curriculum is structured into five progressive stages, advancing from codebase cartography to automated diagnostic correction, test harness construction, multi-agent scaling, and quality gates.

Stage	Module Title	AGY Tools & Focus	Hands-on Deliverable
Stage 1	Codebase Cartography & Exploration	grep_search, find_by_name, view_file, Symbol Indexing	Complete architectural and state lifecycle map of Node Hierarchy Manager.
Stage 2	Diagnostic & Error Correction Workflows	replace_file_content, Compiler Loops, Planning Mode	Surgical resolution of recursive cycle bugs & TypeScript strictness errors.
Stage 3	Automated Test Harness & Verification	run_command, Vitest, React Testing Library, MSW	Automated unit and integration test suite with Red-Green-Refactor loops.

Stage 4	Parallel Subagents & Multi-Modal Auditing	invoke_subagent, manage_task, Branch Workspaces	Concurrent Security, Performance, and Test Coverage auditing team.
Stage 5	Guardrails, Custom Skills & Continuous Gates	AGENTS.md, Custom Skills, Slash Commands (/boost, /goal)	Zero-regression quality gate preventing breaking changes automatically.

Module 1: Codebase Cartography & Exploration Mastery

Objective: Learn how AGY reads, indexes, and queries large existing codebases without flooding context windows or hallucinating dependencies.

Key Techniques & AGY Mechanics:

- **Precision File Scoping:** Using `find_by_name` with glob filters (e.g. `*.tsx`, `*.sql`) to locate files across dense project structures.
- **High-Speed Pattern Locating:** Using `grep_search` with `ripgrep` regular expressions to map state hooks (e.g., `useState`, `useEffect`, `supabase.from`) without reading entire files.
- **Slice-Based Code Inspection:** Using `view_file` with precise line offsets (`StartLine / EndLine`) to inspect critical functions without burning token context.
- **Mental Model:** AGY acts as an active cartographer. Pavel learns to prompt AGY with specific architectural questions rather than asking it to 'read everything'.

■■ Hands-on Lab 1: Mapping the Node State Lifecycle

Task for Pavel: Instruct AGY to trace how a node's title is modified in `NodeDetailsModal.tsx`, passed through `NodeTree.tsx`, and saved to Supabase via `supabase_migration_create_node_function.sql`.

Target AGY Prompt: *'Trace the entire execution flow when a user renames a node in NodeDetailsModal. Identify the React state update, the Supabase RPC call, and the SQL trigger that executes.'*

Expected Output: AGY returns an exact call graph with file line numbers and data contracts.

Module 2: Diagnostic & Automated Error Correction Workflows

Objective: Master the diagnostic loop where AGY identifies root causes, plans surgical fixes, and verifies code with zero regression.

The 4-Step Diagnostic Protocol:

1. **Error Ingestion:** Capturing compiler output (`tsc -b`), ESLint reports, or browser runtime stack traces.
2. **Root Cause Hypothesis:** Formulating why the failure occurred (e.g. state mutation in render cycle, undefined Supabase response).
3. **Planning & Surgical Diff:** Creating an `implementation_plan.md` and applying minimal, non-destructive edits via `replace_file_content`.
4. **Closed-Loop Verification:** Re-executing the linter/compiler command immediately to guarantee that the error code is 0.

Bug Class	Symptom in Node App	AGY Correction Technique
Recursive Cycle Exception	Stack overflow when rendering tree with cyclical parent-child relationships.	AGY adds a visited: Set<string> cycle guard in tree traversal with pure functional memoization.
Supabase Cascade Delete Race	Deleting a parent node leaves orphan children or triggers foreign key constraint errors.	AGY refactors SQL migration with ON DELETE CASCADE and updates client-side optimistic UI state.
TypeScript Strict Null Mismatch	Object is possibly 'null' when parsing NotebookLM metadata.	AGY introduces TypeScript narrowing guards (<code>isNodeMetadata(x)</code>) and interface refinement.

■■ Hands-on Lab 2: Injecting & Surgically Fixing a Complex Bug

Task for Pavel: Introduce an intentional TypeScript error or broken async handler in `CurationModal.tsx`. Then prompt AGY to identify the failure via `npm run build`, inspect the exact line error, propose an `implementation_plan.md`, execute the fix with `replace_file_content`, and verify zero compiler errors.

Key Takeaway: Observe how AGY systematically validates compiler output before declaring the task done.

Module 3: Automated Testing & Closed-Loop Verification

Objective: Transition from manual testing to autonomous, test-driven development (TDD) pipelines where AGY writes, runs, and enforces test suites.

The Test-Driven Agentic Architecture:

- **Test Harness Setup:** Prompting AGY to install and configure Vitest, React Testing Library, and jsdom for the Vite React 19 app.
- **Mocking External Boundaries:** Directing AGY to build clean mock handlers for the Supabase client and NotebookLM REST endpoints.
- **The Red-Green-Refactor Loop:** Instructing AGY to write a failing test first (Red), implement the component feature (Green), and optimize performance (Refactor).
- **Pre-Flight Verification:** Enforcing that all automated tests (npm test) and linters pass before AGY delivers the final walkthrough.md.

■■ Hands-on Lab 3: Building a Vitest Tree Hierarchy Test Suite

Task for Pavel: Prompt AGY to create src/components/___tests___/NodeTree.test.tsx covering:

1. Rendering a 3-level nested node hierarchy correctly.
2. Handling empty state when no nodes are returned from Supabase.
3. Triggering node selection and modal opening on click.
4. Edge Case: Preventing circular parent-child loops from locking the browser thread.

Execution: AGY runs npx vitest run, detects any unhandled edge case, repairs the code, and achieves 100% passing status.

Module 4: Parallel Subagents & Multi-Agent Auditing

Objective: Scale Pavel's engineering productivity by delegating independent tasks to specialized parallel subagents.

Subagent Role	Specialization & System Directives	Application Impact
Security & Perms Auditor	Audits Supabase SQL migrations, RLS policies, and API key exposure risks. Checks AGENTS.md compliance.	Ensures zero unauthenticated data leaks across multi-tenant node hierarchies.
Performance Profiler	Inspects React render cycles in NodeTree.tsx, identifies unnecessary re-renders, and adds useMemo.	Guarantees smooth 60fps rendering even with 10,000+ hierarchical nodes.
Test Coverage Writer	Scans all 15 modal components in src/components/ and drafts comprehensive unit test fixtures.	Raises overall test coverage from 0% to >85% across all modals in parallel.

■■ Hands-on Lab 4: Launching a 3-Agent Parallel Audit

Task for Pavel: Prompt AGY to spawn three concurrent subagents using invoke_subagent to simultaneously review: (1) Security rules in supabase_migration_auth_v1.sql, (2) React memoization in Nodeltem.tsx, and (3) ESLint strictness in eslint.config.js.

Learning Outcome: Pavel learns how to review synthesized findings from multiple subagents without context overload.

Module 5: Guardrails, Custom Skills & Continuous Quality Gates

Objective: Encode project-specific engineering rules, safety guardrails, and custom domain skills so AGY acts as an elite, compliant senior engineer on every interaction.

The Hierarchy of Customization in AGY:

- **Workspace Operational Rules (AGENTS.md):** Unbreakable global policies (e.g. 'Never delete files without confirmation', 'All outputs must be PDF', 'Always run linters before completion').
- **Domain-Specific Skills (.agents/skills/):** Modular capability packages containing scripts, reference docs, and executable workflows (e.g. gemma-eval, supabase-migrator).
- **Slash Commands:** Quick shortcuts like /boost (multi-perspective reasoning), /goal (unsupervised task execution), /grill-me (architectural interview), and /learn (behavior persistence).

■ Hands-on Lab 5: Authoring a Custom Verification Skill

Task for Pavel: Create a new skill .agents/skills/node-tree-validator/ that equips AGY with an automated script to validate hierarchical node data integrity, check foreign key constraints, and run test suites with a single prompt.

Result: Any future developer or agent working in this repo can invoke the skill to perform instant automated validation.

4. Pavel's Practical Prompt Playbook & Cheatsheet

To get maximum precision and zero hallucination from AGY, use these field-tested prompt templates tailored for the Node Hierarchy Manager.

Scenario	High-Precision Prompt Template for Pavel
Bug Diagnosis & Root Cause Analysis	'Investigate the runtime error when rendering NodeTree.tsx. Use grep_search to find all references to activeNodeId, view the relevant slice in NodeTree.tsx, isolate the root cause without editing, and prepare an implementation_plan.md for my approval.'
Test-Driven Bug Fix (TDD)	'Write a failing Vitest unit test in src/components/__tests__/NodeItem.test.tsx that reproduces the state desync on node rename. Verify the test fails via run_command. Then apply a targeted fix using replace_file_content and verify the test passes.'
Comprehensive Pre-Flight Verification	'Run a full quality audit on the project: execute npm run build and npm run lint. If any TypeScript type errors or ESLint warnings occur, fix them cleanly with replace_file_content and document all changes in walkthrough.md.'
Multi-Modal Refactoring with Subagents	'Spawn a research subagent to audit all 15 modals in src/components/ for duplicate Supabase query logic. Have the subagent report a consolidated abstraction plan before we modify any source code.'

■ Graduation Capstone Milestone for Pavel

By completing this 5-stage learning journey, Pavel will possess full mastery over Google Antigravity (AGY). Pavel will be able to fearlessly navigate complex enterprise codebases, delegate massive refactors to parallel subagent swarms, enforce unbreakable safety policies via AGENTS.md, and orchestrate automated test verification pipelines with complete confidence.